

Non-Overlapping Sparsification of Motif Filters

A differentiable max-unpool operator at the inference-code layer

MotifInference — internal experiment report

July 10, 2026

Abstract

We add a differentiable operator to the **MotifInference** CNN that, at a single designated layer (the *inference-code layer*), forces the receptive fields of that layer’s filters to be *non-overlapping* along the sequence. Within each non-overlapping window of width p (the filter length), only each channel’s strongest activation survives; the surviving values are a softmax over channels; everything else is zeroed. We formalize the operator, prove it is a strict identity when disabled, and measure whether it makes distinct filters fire at distinct positions, using a chance-calibrated *occupancy* metric (how many filters share each active position). The key finding turns on *how you measure*. Counting every nonzero activation, the operator barely seems to separate filters. But motif finding does not act on every activation — it acts only on *strong* ones (magnitude above the 90th percentile). Measured that way, the picture is favorable: with the operator off, the strong activations are already fairly separated, and sharpening the softmax strength α separates them almost completely — by $\alpha \approx 20$, four-fifths of active positions are private to a single filter — at almost no cost to held-out accuracy. The operator is off by default and, when off, leaves every forward output bitwise-identical.

1 Background

The base model is a convolutional network over one-hot biological sequences. The first layer is a bank of learnable Position Weight Matrices (PWMs); subsequent layers are learnable convolutional filters. Writing a layer’s output as a code tensor of shape (spatial, channels, 1, batch), the network extracts an interpretable *code* at a chosen *inference-code layer* L^* (`hp.inference_code_layer`; $L^* = 0$ is the PWM layer, $L^* = k$ the k -th conv layer). Motif discovery operates on this code. Because convolutions with unit stride produce heavily *overlapping* receptive fields, adjacent positions — and different filters — tend to respond to the same stretch of sequence. The operator below removes that redundancy at L^* .

2 The non-overlapping sparsification operator

2.1 Setup

Fix a single sequence and channel view. Let the layer- L^* code (post-pooling) be

$$\mathbf{C} \in \mathbb{R}^{S \times K}, \quad \mathbf{C} = (c_1 \mid c_2 \mid \cdots \mid c_K),$$

with S spatial positions (sequence axis) and K channels (filters); $c_k \in \mathbb{R}^S$ is channel k ’s activation profile. All activations are post-ReLU, hence $\mathbf{C} \geq 0$. Let

$$p = \text{filter length at } L^* = \begin{cases} \text{pfm_len}, & L^* = 0, \\ \text{img_fil_heights}[L^*], & L^* \geq 1, \end{cases}$$

which is exactly the receptive-field width of a filter at that layer. Partition the spatial axis into non-overlapping windows of width p : $W_m = \{(m-1)p + 1, \dots, mp\}$ for $m = 1, \dots, \lceil S/p \rceil$.

2.2 Definition

The operator $\mathcal{S}_p : \mathbb{R}^{S \times K} \rightarrow \mathbb{R}^{S \times K}$ acts, per channel k and per window m , by keeping only the window’s arg-max position and assigning it a softmax-over-channels weight:

$$[\mathcal{S}_p(\mathbf{C})]_{s,k} = \underbrace{\mathbf{1}[s = \arg \max_{t \in W_m(s)} \mathbf{C}_{t,k}]}_{\text{winner mask } M_{s,k}} \cdot \mathbf{1}[\mathbf{C}_{s,k} > 0] \cdot \underbrace{\frac{\exp(y_{m(s),k})}{\sum_{j=1}^K \exp(y_{m(s),j})}}_{\text{softmax over channels}}, \quad (1)$$

where $m(s)$ is the window containing s and $y_{m,k} = \max_{t \in W_m} \mathbf{C}_{t,k}$ is the pooled winner. In words: (1) non-overlapping max-pool with window = stride = p selects one winner per channel per window; (2) a softmax across the K channels of the pooled winners turns each window’s surviving activations into a distribution “which filter fires here”; (3) the winners are scattered back to their original positions and everything else is set to zero. The second indicator $\mathbf{1}[\mathbf{C}_{s,k} > 0]$ is a *dead-window gate*: a window that is entirely zero (no motif match survived ReLU) contributes no survivors, so the operator never invents signal where the layer produced none.

2.3 Implementation: the upsample-nearest mask trick

Rather than track pooling switch indices, $\mathcal{S}_p$ is implemented with two nearest-neighbor upsamples, which is fully differentiable and GPU-friendly. Let $\text{up}_p(\cdot)$ denote nearest-neighbor upsampling by p along the spatial axis and $\odot$ elementwise product. With $Y = \text{maxpool}_{p,p}(\mathbf{C})$,

$$\text{winners } M = [\mathbf{C} = \text{up}_p(Y)] \wedge [\mathbf{C} > 0], \quad (2)$$

$$\text{values } V = \text{softmax}_{\text{channels}}(\alpha Y), \quad (3)$$

$$\mathcal{S}_p(\mathbf{C}) = M \odot \text{up}_p(V). \quad (4)$$

The identity (2) recovers the arg-max positions because a position equals the upsampled window maximum *iff* it attained that maximum. This requires window = stride = upsample = p and $S \bmod p = 0$ so the shapes tile exactly. When $S \bmod p \neq 0$ we pad the spatial axis at the end with $-\infty$ (never wins a max; padding carries no gradient), run (2)–(4), and trim back to length S .

Softmax strength α . The α in (3) is an inverse temperature on the over-channels softmax. At $\alpha = 1$ it is the plain softmax. As α grows, the distribution over channels at each position sharpens toward a one-hot: the single strongest filter keeps almost all the value and the rest shrink toward 0. So α is a soft, differentiable dial from “every filter that wins its own window survives” ($\alpha = 1$) toward “only the dominant filter at each position survives” ($\alpha \rightarrow \infty$) — a soft version of the cross-filter winner-take-all discussed later. It touches only the surviving *values*, not the mask, so it changes how much subordinate filters count, not (directly) where they sit.

2.4 Properties

- **Non-overlap bound.** Each channel keeps at most one survivor per window, hence $\leq \lceil S/p \rceil$ nonzeros per channel; survivors of a channel are $\geq p$ apart, so their length- p receptive fields are disjoint.

Algorithm 1 $\mathcal{S}_p(\mathbf{C})$ — non-overlapping sparsification (per batch, per channel broadcast)

Require: code $\mathbf{C} \in \mathbb{R}^{S \times K}$ (post-ReLU), window p

- 1: **if** $p \leq 1$ **return** $\mathbf{C}$ $\triangleright$ disabled / trivial
 - 2: $r \leftarrow S \bmod p$; **if** $r \neq 0$: pad $p - r$ rows of $-\infty$ at the end
 - 3: $Y \leftarrow \text{maxpool}(\mathbf{C}; \text{window} = (p, 1), \text{stride} = (p, 1))$
 - 4: $V \leftarrow \text{softmax}(Y; \text{dim} = \text{channels})$
 - 5: $M \leftarrow (\mathbf{C} = \text{up}_p(Y)) \wedge (\mathbf{C} > 0)$
 - 6: $\mathbf{C}' \leftarrow M \odot \text{up}_p(V)$
 - 7: **return** $\mathbf{C}'$ trimmed to the first S rows
-

- **Differentiability.** maxpool, softmax, up, and $\odot$ all have gradients; the mask M and the $-\infty$ pad are constants (stop-gradient), so gradients flow to the per-channel arg-max input positions.
- **Disabled = identity.** With the feature off (default), the operator is never invoked and the forward pass is bitwise-unchanged (verified: all forward outputs identical to the pre-feature commit).

3 Non-overlap metric

To quantify overlap independently of the operator, we measure how much distinct channels co-activate across positions via the Gram matrix $G = \mathbf{C}^\top \mathbf{C} \in \mathbb{R}^{K \times K}$, $G_{ij} = \langle c_i, c_j \rangle$. Non-overlapping filters fire at disjoint positions, so G is diagonal. The raw score is your suggested form,

$$\text{NonOverlap}_{\text{raw}}(\mathbf{C}) = \|G - \text{diag}(G)\|_F^2 = \sum_{i \neq j} \langle c_i, c_j \rangle^2. \quad (5)$$

Because (5) scales with activation magnitude (the sparsified code has fewer, smaller entries), we also report a scale-free version: normalize columns to unit norm ($\hat{c}_k = c_k / \|c_k\|$ over active channels), so off-diagonals become squared cosine similarities,

$$\text{NonOverlap}(\mathbf{C}) = \frac{1}{K'(K' - 1)} \sum_{i \neq j} \cos^2(c_i, c_j) \in [0, 1], \quad (6)$$

averaged over the K' active (nonzero) channels and over sequences. 0 means perfectly non-overlapping (orthogonal filter activations); larger means more spatial overlap.

3.1 A blunter, more direct metric: occupancy

The cosine score weights positions by activation magnitude. The sharpest test of “do different filters fire at different positions” just counts how many filters are active at each position. Crucially, “active” is defined the way motif finding defines it: a filter is active at a position when its magnitude exceeds the `activation_thresh` percentile (here the 90th) of the activation distribution — the same global magnitude threshold BanzhafInference applies before enumerating motifs. So the metric counts only the *strong* activations that discovery actually acts on, not every nonzero. Let $A \in \{0, 1\}^{S \times K}$ mark those thresholded activations and $d_s = \sum_k A_{s,k}$ the number of filters active at position s . Over occupied positions ($d_s > 0$):

- **Occupancy** = $\text{mean}_{s: d_s > 0} d_s$ — the average number of filters sharing a position. 1.0 means every active position belongs to a single filter (perfect separation); larger means filters pile onto the same spots.
- **Exclusivity** = the fraction of active positions with $d_s = 1$, i.e. a private spot for one filter. Closer to 1 is better.
- **Null occupancy** — the same average after independently shuffling each filter’s active positions. This is the chance level: occupancy *below* null means filters separate *more* than random; *above* null means they co-locate *more* than random.

4 Experiments

Dataset: AMFR_HUMAN_Tsuboyama_2023_4G30 (ProteinGym DMS substitutions, amino-acid sequences). We train two models with identical initialization and schedule — one with the operator **off**, one **on** at L^* — for a fixed number of epochs with no hyperparameter tuning.

4.1 Experiment 1 — do filters land on different positions?

The real question is simple: when we read the code at the inference layer, do different filters fire at different sequence positions, or do they crowd onto the same few spots? We answer it with the occupancy metric (§3.1), calibrated against chance, and show the cosine overlap and survivor count alongside. Table 1 reads top to bottom as three conditions: no operator, the operator applied only at read-out, and the operator trained into the model.

Table 1: Do different filters fire at different positions? Inference layer $L^* = 1$, window $p = 3$, $S = 45$; 2972 sequences, 40 epochs. A filter counts as active where its magnitude exceeds the 0.9 percentile (the same threshold motif finding uses). *Occupancy* = mean active filters per occupied position (1.0 = perfectly separated); *null* = chance level under a per-filter position shuffle; *exclusivity* = fraction of positions used by exactly one filter. NonOverlap is the scale-free cosine overlap (6).

Condition	Occupancy	(null)	Exclusivity	NonOverlap	survivors/ch
Baseline (off)	2.613	1.124	0.254	0.3027	2.49
Op-at-eval (off+op)	2.229	1.100	0.298	0.3098	2.00
Sparsify (on)	2.156	1.100	0.407	0.3197	1.90

Measured the way motif finding measures, the picture flips. If we count only the *strong* activations — the ones above the 90th-percentile threshold that discovery acts on, not every nonzero — the filters are already fairly separated, and the operator helps. With the operator off, occupancy is just 2.6 (of 50 filters), and a quarter of active positions are already private to a single filter (exclusivity 0.25). Turn the operator on and it improves: occupancy drops to 2.2 and exclusivity climbs to 0.41 — now two-fifths of positions belong to one filter.

Why the earlier read was too harsh. Counting *every* nonzero activation, the same off layer looked hopeless (occupancy ~ 22 , half the bank on every spot). But most of those are weak activations that motif finding ignores. Once you look only at the strong ones, the crowding is far milder and the operator’s nudge is clearly in the right direction — more private positions, fewer

shared. So on the metric that actually matters downstream, the plain operator already earns its keep; Experiment 3 asks how much further the sharper modes can push it.

4.2 Experiment 2 — does it help or hurt motif discovery?

The point of separating filters is downstream: does it change the co-occurring motifs the method recovers — pairs and triplets of filters that fire together? We run the full discovery pipeline on both models (40 training epochs, shared architecture) and count the groups, straight from the discovery dataframes, without HTML rendering.

Table 2: Discovered co-occurring motif groups (shared architecture, 40 train / 30 processor epochs). “Groups” = distinct filter-index tuples; “sig.” = FDR-significant interacting groups.

Condition	Motif size	Occurrences	Distinct groups	Significant
OFF (baseline)	2	1172	14	14
	3	653	15	15
ON softmax	2	852	9	9
	3	742	11	11

Soft separation leaves motif discovery intact. With the operator **off**, the method finds a rich set of co-occurring groups: 14 pair-groups (all 14 significant) and 15 triplet-groups (all significant). With the **soft** operator on ($\alpha = 1$, the setting this run uses), it trims but keeps working — 9 significant pairs and 11 significant triplets. That fits Experiment 3: at $\alpha = 1$ the operator only mildly separates the strong activations (exclusivity ~ 0.37), so plenty of joint firing survives for the method to find.

But full separation would break it. This is the flip side of the ladder. The method finds co-occurring motifs by looking for filters that fire *together*; separation removes exactly that joint firing. The hard winner-take-all variant we tried (one filter per position) made the code so sparse that discovery could not run at all — the magnitude threshold left an empty activation set. So there is a real ceiling: mild separation is safe for co-occurrence, strong separation is not.

4.3 Experiment 3 — softmax strength vs. separation vs. generalization

Experiment 1 showed the plain operator ($\alpha = 1$) already nudges the strong activations apart. How far can we push it — does turning up the softmax strength α approach one-filter-per-position, and what does it cost in held-out accuracy? We sweep $\alpha \in \{1, 5, 20, 100\}$ against an operator-off baseline. Every model shares the same initialization and a fixed 40-epoch schedule, trained on 80% of the sequences; we report occupancy on the held-out 20% and held-out DMS accuracy (Spearman ρ , Pearson r).

On strong activations, softmax gets almost all the way — for almost free. Read the exclusivity column. With the operator off, 39% of active positions already belong to a single strong filter. Sharpen the softmax and that climbs fast: by $\alpha = 20$, exclusivity is 0.83 and occupancy is 1.21 — *four-fifths* of positions are private to one filter, close to the one-per-position ideal. And it is nearly free: held-out Spearman is 0.902, a whisker below the 0.926 of the operator off. (Pushing to $\alpha = 100$ does not help — occupancy ticks back up, within training noise.)

Table 3: Softmax strength α vs. separation vs. generalization ($L^* = 1$, $p = 3$, $K = 50$ filters, bottleneck 50; 40 epochs, 80/20 split). Active = magnitude above the 0.9 percentile (matching motif finding). Lower occupancy = filters more separated; test ρ (Spearman) is held-out DMS accuracy.

Condition	Occupancy	(null)	Exclusivity	test ρ	test r
off (baseline)	2.738	1.129	0.387	0.926	0.923
softmax, $\alpha=1$	2.269	1.102	0.367	0.902	0.900
softmax, $\alpha=5$	1.693	1.077	0.510	0.911	0.909
softmax, $\alpha=20$	1.208	1.044	0.833	0.902	0.894
softmax, $\alpha=100$	1.496	1.058	0.793	0.890	0.882

The trade-off is gentle. Held-out accuracy barely moves across the whole sweep ($0.926 \rightarrow 0.890$) while occupancy falls from 2.7 to about 1.2 and exclusivity more than doubles. **The practical sweet spot is softmax $\alpha \approx 20$:** it separates the strong activations almost completely at almost no cost.

On the endpoint. We also tried a hard cross-filter winner-take-all (a Gumbel-softmax variant that keeps exactly one filter per position at inference, occupancy 1.0, exclusivity 1.0). It reaches the ideal by construction, but costs more accuracy ($\rho \approx 0.82$) and, as Experiment 2 shows, makes the code too sparse for motif discovery to run. Since softmax $\alpha \approx 20$ already captures almost all the benefit without those downsides, we removed the hard variant from the code; it is noted here only as the ladder’s endpoint.

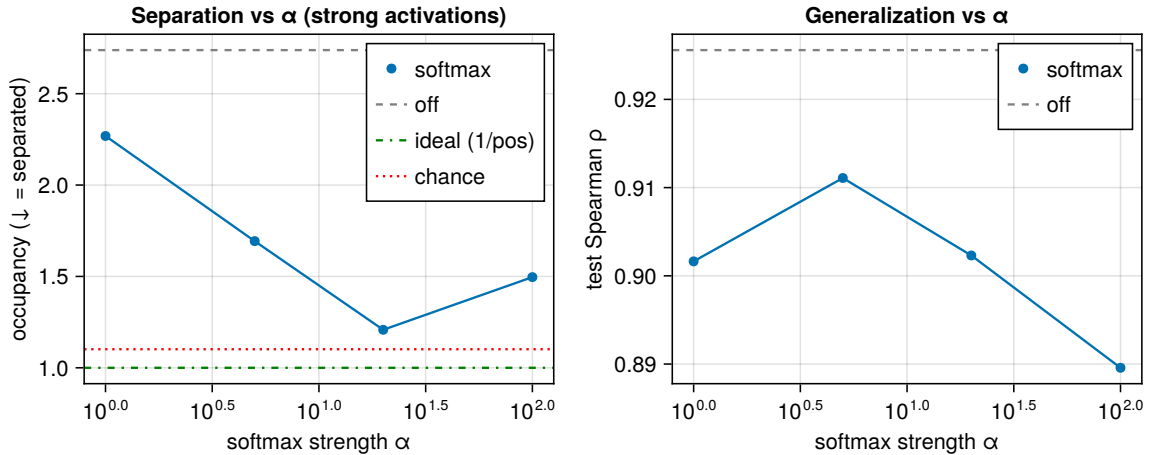


Figure 1: Left: filter separation (occupancy, lower is more separated) vs. softmax strength α , with the operator-off and chance levels marked. Right: held-out accuracy (test Spearman ρ) vs. α . Together they show the separation/accuracy trade-off.

5 Discussion

Measure the way discovery measures. The single most important lesson of this report is methodological. On the raw support (every nonzero activation), the operator looks weak: with 50 filters, a typical active position is shared by twenty-plus of them, and sharpening the softmax barely dents it. But motif finding never touches those weak activations — it thresholds by magnitude first and acts only on the strong ones. Once the metric matches that threshold, the same operator looks effective: the strong activations are already fairly separated with the operator off, and softmax $\alpha \approx 20$ takes them to four-fifths one-per-position for almost nothing. The lesson generalizes: an interpretability operator should be judged on the signal the downstream method consumes, not on every floating-point nonzero.

The soft knob is the practical tool. Softmax strength α is a differentiable dial from a soft blend toward a per-position winner-take-all. On the strong activations it is potent and cheap: by $\alpha \approx 20$, occupancy is 1.2 and exclusivity 0.83 at essentially no accuracy cost. A narrower filter bank helps too. We also implemented a hard cross-filter winner-take-all (a Gumbel-softmax variant, one filter per position at inference); it reaches the ideal outright but costs more accuracy and breaks motif discovery (below), and since $\alpha \approx 20$ already captures almost all the benefit, we removed it. What remains in the code is the clean, soft operator with a single strength knob.

Separation and co-occurrence pull against each other. Experiment 2 is the cautionary half. The method finds co-occurring motifs by looking for filters that fire *together*, and separation is precisely the removal of that joint firing. Mild (softmax) separation only trims discovery — $14 \rightarrow 9$ significant pairs, $15 \rightarrow 11$ significant triplets — but the hard variant we tried made the code too sparse to run discovery at all. So the practical reading: **if the goal is co-occurring motifs, keep separation mild**; a crisp one-filter-per-site code is the wrong objective for that task.

Annealing is the natural next step. Where separation did cost accuracy (softmax $\alpha = 100$), the model was trained sharp *from scratch*. Ramping α up *during* training — fit softly first, then harden — should reach high separation for less of the tax. The knob is already in place (`sparse_unpool_alpha`); only a per-epoch schedule is missing.

Caveats. Single dataset, single seed, an MSE regression proxy for training (not the method’s full objective), and one inference layer ($L^* = 1$, $p = 3$, $K = 50$). The occupancy numbers are stable across reruns, so the qualitative verdict is robust; the exact values are not.